UNIVERSIDAD DE PAMPLONA

Sede Villa del Rosario · Facultad de Ingenierías y Arquitectura

Ingeniería de Sistemas · Ingeniería del Software III · Código 167420

Proyecto Integrador · Semanas 9 y 10

Metodologías de desarrollo y su impacto en la arquitectura

Chronos, Sistema de Turnos: comparativo «clásico vs. actual» y backlog arquitectónico por sprints

Parte	Entregable	Secciones
I	**Comparativo «clásico vs. actual»** (semana 9)	1. Mapeo a RUP · 2. Tres cambios · 3. Comparación crítica · 4. Arquitectura hexagonal
II	**Backlog arquitectónico por sprints** (semana 10)	5. Deuda técnica · 6. Spikes · 7. Backlog priorizado · 8. Dos sprints · 9. Terminado · 10. Tablero
Anexo	Decisiones de arquitectura y referencias	A. Decisiones a registrar como ADR · Referencias

Grupo	5
Integrantes	Andrés Camilo León Calderón · Jhorman Stiven Hernández Durán
Docente	Carlos Arturo Barrientos Suárez
Repositorios	`Jhorm4nnn/Chronos-Domain-Service` (commit 75be86e) · `Jhorm4nnn/Chronos-AuthCore` (commit d35238c)
Fecha	9 de noviembre de 2026

Fuentes y convenciones

Todo dato de este documento sale de una fuente verificable: el código de los dos repositorios, el product backlog (`docs/PRODUCT BACKLOG SISTEMA DE TURNOS.xlsx`), la hoja del Taller 9 (`docs/taller9/`) y los documentos anteriores del proyecto (`docs/antecedentes/`). Los conteos de archivos se hicieron con *git grep -l* sobre el código real.

Las historias se citan con la numeración del product backlog vigente (HU-1 a HU-14). La guía del docente, el README y la hoja del Taller 9 usan la numeración HU-8 a HU-12. Equivalencias: HU-8 de la guía = HU-1 y HU-9 del backlog; HU-9 = HU-2; HU-10 = HU-3; HU-11 = HU-4 y HU-10; HU-12 = HU-5, HU-11 y HU-12. Unificarla es la deuda D-11.

PARTE I · ENTREGABLE DE LA SEMANA 9

1. El proyecto visto con la lente de RUP

El curso no se trabajó formalmente con RUP. Usamos sus cuatro fases y sus hitos como herramienta de análisis para ubicar cuándo tomamos cada decisión de arquitectura y qué evidencia la respalda. La figura 1 muestra el semestre con las fechas reales del repositorio.

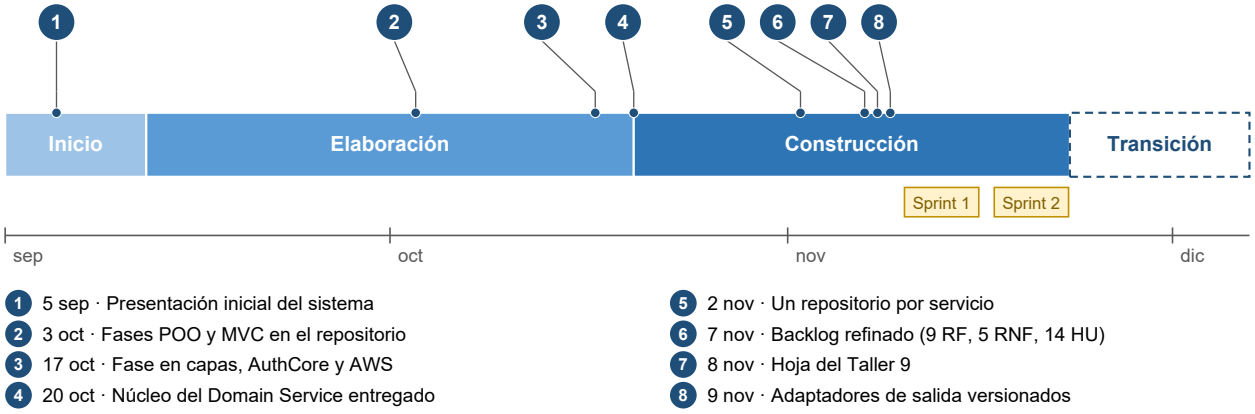


Figura 1. El semestre de Chronos leído con las fases de RUP. Los hitos son fechas reales de los repositorios y documentos; los límites entre fases son aproximados. Los sprints corresponden a la parte II.

Fase y periodo	Qué hicimos	Evidencia en el repositorio	Hito de salida
Inicio hasta el 12 sep	Problema (citas en cuaderno, choques de horario), alcance (agenda sin cruces) y primeros 5 RF y 3 RNF.	`SISTEMA DE TURNOS INTRODUCCION.pdf` (5 sep) y backlog de la Revisión 1, en `docs/antecedentes/`.	**Cumplido.** El docente validó el alcance: «5 RF y 3 RNF - conteo correcto» (Hoja 1 del backlog).
Elaboración 12 sep – 20 oct	Vistas 4+1; el mismo servicio en cuatro arquitecturas (POO, MVC, capas, hexagonal); AuthCore hexagonal con BCrypt y JWT; login integrado desde la fase en capas.	Documento de arquitectura lógica; código de las fases y de AuthCore en el repositorio original (3, 10 y 17 oct); plantilla AWS en `Chronos-AuthCore/docs/aws/`.	**Cumplido con reservas.** Arquitectura ejecutable: núcleo del Domain Service (20 oct) con 66 pruebas de dominio y aplicación, según la guía del docente. Sin probar la integración JPA y REST.
Construcción 20 oct – hoy	Domain Service con las HU de turnos; un repositorio por servicio (2 nov); backlog refinado a 9 RF, 5 RNF y 14 HU (7 nov); Taller 9 (8 nov).	Los dos repositorios; product backlog en `docs/`; hoja del Taller 9 en `docs/taller9/`.	**Pendiente.** No hay capacidad operativa: nunca se ejecutó *mvn verify* completo ni el build de AuthCore (deuda D-01).
Transición pendiente	Guía de uso, colección de pruebas y despliegue conjunto de los dos servicios.	README de ambos repositorios, `requests.http` y guía del Domain Service.	**No alcanzado.** Falta desplegar y demostrar los dos servicios juntos.

Qué muestra el mapeo. La arquitectura se fijó al cerrar Elaboración y con código que se ejecuta, como pide RUP: dominio, puertos y validación local del JWT. Pero un riesgo técnico pasó a Construcción sin retirarse: nunca verificamos la integración completa (pruebas Spring, persistencia JPA, build de AuthCore). RUP habría exigido probarla antes del hito.

2. Tres cambios trazados en cascada y en RUP

En el Taller 9 elegimos un cambio de cada tipo y lo medimos sobre el código real (commit 7be471f). La tabla resume la hoja del taller. El costo en cascada y en RUP se mide en fases reabiertas y documentos rehechos. El costo en código se mide en archivos y capas. No usamos horas ni dinero.

Cambio pedido	En cascada	En RUP	Archivos tocados (hexagonal)	Qué lo absorbió
1 · Regla de negocio. 10 minutos de descanso entre turnos seguidos del mismo profesional.	Reabre 2 fases (Requisitos y Diseño): el criterio de HU-3 permite turnos pegados. Se repite la fase de Pruebas.	Iteración de Construcción en curso. Ningún hito se mueve.	**5** = 1 de producción (`ValidadorAgenda`) + 3 pruebas + 1 doc. 1 capa de producción.	El dominio: la regla de cruce tiene un solo dueño.
2 · Infraestructura. Pasar de secreto compartido (HS384) a par de claves (RS256).	Reabre 2 fases (Diseño de seguridad e Implementación) en los dos servicios. Se descubre al desplegar.	Riesgo propio de Elaboración, cuyo hito se cerró con HS384. Obliga a actualizar la línea base de arquitectura en la siguiente iteración.	**7** = 3 de producción (`JwtTokenValidator`, `JwtProperties`, `SecurityConfig`) + 2 pruebas + 2 config. y docs. +5 en AuthCore.	El adaptador de seguridad: aplicación y dominio no cambian.
3 · Alcance. Marcar un turno como ATENDIDO o NO_ASISTIÓ, por API y por consola.	Reabre 4 fases (Requisitos, Diseño, Implementación y Pruebas) y obliga a renegociar el alcance.	Historia nueva para la siguiente iteración. Ningún hito se mueve.	**16** = 8 de producción en 4 capas + 6 pruebas + 2 docs.	El dominio (`Turno`, `EstadoTurno`) y el puerto `TurnoUseCase`, compartidos por REST y consola.

> **Actualización del 9 de noviembre al conteo del cambio 3**
>
> La hoja del taller dejó la capa de Salida «por revisar» porque los adaptadores de salida no estaban en GitHub: la regla `out/` del `.gitignore` excluía `application/port/out` e `infrastructure/adapters/out`. Ya están versionados (commit 75be86e), y el código confirma la sospecha del taller: `TurnoRepositoryAdapter` (líneas 68 y 69) filtra con `estado = AGENDADO` cuando no se piden cancelados. Con el cambio, los turnos ATENDIDO desaparecerían de la consulta. El cambio 3 sube de 15 a 16 archivos.

Lectura de costos. La metodología no cambia el número de archivos, porque el código es el mismo. Cambia cuándo se descubre el cambio y cuánto trabajo hay alrededor. En cascada se reabren en total 8 fases para los tres cambios. En RUP no se reabre ninguna: dos cambios entran en una iteración y uno (RS256) actualiza la línea base de arquitectura. El cambio 2 se lo habría ahorrado RUP si la seguridad entre los dos servicios se hubiera tratado como riesgo y retirado en Elaboración.

3. Comparación crítica con la forma en que trabajamos

No trabajamos ni en cascada ni con RUP. Nuestro proceso fue iterativo y las revisiones del docente hicieron de hitos. Construimos el mismo servicio cuatro veces (POO, MVC, capas y hexagonal) sobre las mismas historias: eso se parece a las iteraciones de Elaboración de RUP. Pero cada fase reescribió el código desde cero, y en eso cada fase costó lo que cuesta un ciclo completo de cascada. Lo aceptamos porque el objetivo era comparar arquitecturas.

Aciertos (y su evidencia)	Errores (y cuándo los descubrimos)
Comparar con evidencia. La misma funcionalidad en cuatro arquitecturas permitió medir, no opinar (documento de arquitectura, sección 7).	**Requisitos ambiguos.** «Consultar la API» en RF-02 y tres RNF numerados como RNF-01. Los encontró el docente en su revisión, no nosotros al escribirlos: el síntoma de cascada de descubrir tarde.
Reglas con un solo dueño. Desde la fase en capas la regla de cruce vive en una clase (`ReglaSinCruce`, hoy `ValidadorAgenda`). Por eso el cambio 1 costó un archivo.	**Requisitos con varias acciones.** RF-01, RF-04, RF-05, RNF-01 y RNF-03 mezclaban acciones. Se separaron el 7 de noviembre en 9 RF, 5 RNF y 14 HU.
Revisiones atendidas rápido. Las observaciones del docente se resolvieron el mismo día, con un commit por corrección (4a1c183, e9a8d74 y 182a9f8).	**Secretos en el repositorio.** Una contraseña de MySQL quedó como valor por defecto en el repositorio original, que fue público hasta el 7 de noviembre. AuthCore crea además admin/admin123 al arrancar.
Medir con código. El Taller 9 contó archivos con *git grep* y anotó lo que no podía verificar en lugar de suponerlo.	**Código incompleto sin que nadie lo notara.** Del 2 al 9 de noviembre faltaron en GitHub 31 archivos de puertos y adaptadores de salida. Nada compila el repositorio en cada cambio; lo descubrió el taller.

Aciertos (y su evidencia)	Errores (y cuándo los descubrimos)
Arquitectura verificada. ArquitecturaHexagonalTest (ArchUnit) hace fallar el build si el dominio importa Spring o JPA.	**Sin revisión entre pares.** Los 37 commits del repositorio original fueron directo a main, casi todos desde la web («Add files via upload»). La verificación completa se aplazó: las pruebas Spring nunca se ejecutaron.

Cambios de alcance reales. Entre fases cambiaron los datos del cliente (documento y género pasaron a teléfono y correo), la interfaz (Swing pasó a REST y consola), la persistencia (archivos de texto pasaron a JPA) y la firma del token (HS256 pasó a HS384). Como cada fase reescribía todo, ninguno se absorbió cambiando un adaptador: todos fueron reescrituras. Desde la fase hexagonal ya sí sería posible, como muestra el cambio 2 del taller.

Conclusión. Nuestro proceso se pareció a RUP en los hitos, que fueron las revisiones del docente, y a cascada en cómo descubrimos los errores: en la revisión o al final. Lo que los habría detectado antes es integración continua, revisión entre pares y una definición de terminado. Por eso son la primera meta de la parte II.

4. Conexión con la arquitectura hexagonal

La figura 2 cruza los tres cambios con las capas del Domain Service. Las cifras son archivos reales tocados en cada capa.

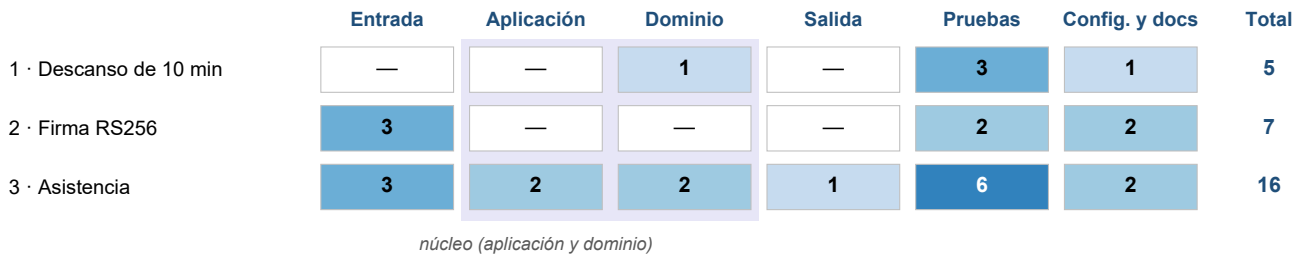


Figura 2. Archivos tocados por capa en cada cambio del Taller 9, sobre el código hexagonal real.

- **Cambio 1, responsabilidad única.** `git grep -l buscarConflicto -- src/main` devuelve dos archivos: `domain/service/ValidadorAgenda.java`, dueño de la regla, y `application/service/TurnoService.java`, que la invoca. REST y consola llegan a la regla por TurnoService, así que el cambio se queda en el dominio.
- **Cambio 2, inversión de dependencias.** Solo `infrastructure/security/JwtTokenValidator.java` verifica firmas. El dominio y la aplicación no importan jjwt (lo prueba ArquitecturaHexagonalTest), así que el cambio de algoritmo no atraviesa el núcleo.
- **Cambio 3, puertos y adaptadores.** `TurnoController` y `TurnosCli` llaman al mismo `TurnoUseCase` (RNF-05, lógica sin duplicar). Hay más archivos porque hay dos puertas de entrada, pero la regla no se duplica. El dolor viene de una regla de negocio filtrada al adaptador de salida (`TurnoRepositoryAdapter`, líneas 68 y 69) y al doble de pruebas (`TurnoRepositoryEnMemoria`, línea 47). El ítem A-07 la devuelve al dominio.

Lo que aprendimos

La metodología decide cuándo descubrimos un cambio y cuánto retrabajo hay alrededor; la arquitectura decide cuántos archivos toca y en qué capas. En Chronos la hexagonal no siempre toca menos archivos (cambio 3), pero deja a cada regla con un solo dueño y hace que las pruebas avisen cuando algo cambia.

PARTE II · ENTREGABLE DE LA SEMANA 10

5. Registro de deuda técnica

Registramos 15 deudas, todas del propio proyecto. Cada una indica dónde se ve en el repositorio. El cuadrante sigue a Fowler (2009) y la prioridad dice cuándo pagarla, no cuánto cuesta.

ID	Deuda	Evidencia	Riesgo	Cuadrante	Prior.
D-01	Pruebas Spring (contrato JPA, unicidad, REST) escritas y nunca ejecutadas; ningún servicio se ha compilado en limpio	README del Domain Service, «Estado honesto»	Fallos de integración sin detectar	Deliberada, prudente	Alta
D-02	Sin integración continua: nada compila ni prueba el código en cada cambio	31 archivos ausentes de GitHub del 2 al 9 nov (75be86e, d35238c)	Código roto o incompleto sin aviso	Inadvertida, prudente	Alta
D-03	AuthCore sin wrapper de Gradle y sin pruebas propias (solo contextLoads, que exige MySQL)	`gradlew` sin `gradle/wrapper/`	No se puede compilar ni verificar en otra máquina	Inadvertida, imprudente	Alta
D-04	Credenciales fijas: admin/admin123 creado al arrancar y escrito en el log; contraseña de MySQL en el historial del repositorio original	`AdminBootstrapRunner`	Acceso indebido	Deliberada, imprudente	Alta
D-05	Esquema de base de datos generado por Hibernate en los dos servicios	`ddl-auto: update` en `application.yml` y `application-prod.yml`	Cambios de esquema sin control	Deliberada, prudente	Alta
D-06	Contraseñas por debajo de HU-1 de AuthCore (exige 6 caracteres; HU-1 pide 8 con letras y números) y registro sin correo	`RegisterRequest`	Contraseñas débiles; HU-1 incumplida	Inadvertida, prudente	Media
D-07	La regla «turno vigente» está escrita como estado = AGENDADO en el adaptador de salida y en el doble de pruebas	`TurnoRepositoryAdapter`, l. 68-69 `TurnoRepositoryEnMemoria`, l. 47	Un estado nuevo rompe la consulta (cambio 3)	Inadvertida, prudente	Media
D-08	Secreto compartido HS384: quien tenga la clave del Domain Service puede fabricar un token ADMIN	`JWT_SECRET_BASE64` en ambos servicios	Escalamiento de privilegios si se filtra	Deliberada, prudente	Media
D-09	Tokens JWT sin revocación, válidos 1 hora	`expiration-ms: 3600000`	Token vigente tras cerrar sesión o quitar un rol	Deliberada, prudente	Media
D-10	Sin medición del tiempo de respuesta (criterio 5 de HU-7)	Ninguna métrica ni cabecera de tiempo	RNF-02 no se puede demostrar	Deliberada, prudente	Media
D-11	Numeración de HU distinta entre documentos	Backlog HU-1..14; guía, README, requests.http y Taller 9 HU-8..12	Trazabilidad confusa	Inadvertida, prudente	Media
D-12	Sin documentación OpenAPI	`pom.xml` y `build.gradle` sin springdoc	Integración manual y propensa a errores	Deliberada, prudente	Media
D-13	Los conflictos de concurrencia responden 409 sin reintento	`RestExceptionHandler` l. 52; @Version y bloqueo pesimista	Error visible ante operaciones simultáneas	Inadvertida, prudente	Baja
D-14	Listados sin paginación	GET de clientes, profesionales y turnos	Lentitud con muchos datos	Deliberada, prudente	Baja
D-15	Plataformas distintas: Java 21, Spring Boot 4.1.1 y Gradle frente a Java 17, Spring Boot 3.5.6 y Maven	`build.gradle` y `pom.xml`	Doble mantenimiento y configuración	Inadvertida, prudente	Baja

Distribución en el cuadrante de Fowler. La mayor parte de la deuda es deliberada y prudente: la aceptamos para avanzar y está anotada. Las dos imprudentes (D-03 y D-04) se pagan en el primer sprint, porque no hubo una decisión consciente detrás.

	Prudente	Imprudente
Deliberada	D-01, D-05, D-08, D-09, D-10, D-12, D-14 (7)	D-04 (1)
Inadvertida	D-02, D-06, D-07, D-11, D-13, D-15 (6)	D-03 (1)

6. Spikes

Cada spike responde una sola pregunta en un tiempo fijo. No entrega funcionalidad: entrega una decisión con evidencia. Si el tiempo se agota, se decide con lo aprendido y se registra como ADR.

ID	Pregunta	Tiempo	Resultado esperado	Decisión que habilita
SP-1	¿Con qué versión de Gradle compila AuthCore (Spring Boot 4.1.1, Java 21) en una máquina limpia, y conviene alinearlo con el Domain Service?	2 h	Wrapper generado y `gradlew build` en limpio; tabla de compatibilidad	Cierra D-03; decide D-15
SP-2	¿Flyway o Liquibase para versionar el esquema en H2 (desarrollo) y MySQL (producción)?	2 h	Prueba de concepto con la tabla de turnos en los dos perfiles	Cierra D-05
SP-3	¿Firmar con RS256, dejando solo la clave pública en el Domain Service, elimina el riesgo de D-08 sin llamar a AuthCore en cada petición?	½ día	Token de AuthCore validado con la clave pública; archivos tocados en los dos servicios	Decide D-08 (cambio 2 del taller)
SP-4	¿Cómo revocar tokens sin perder el carácter sin estado: vida corta con renovación o lista de revocación?	½ día	Comparación de las dos opciones con su costo en archivos	Decide D-09
SP-5	¿Cómo demostrar RNF-02: filtro con cabecera X-Tiempo-Ms (como en la fase hexagonal anterior) o métricas de Actuator?	2 h	Medición de las cuatro operaciones con 100 turnos cargados	Cierra D-10 y HU-7
SP-6	¿springdoc-openapi documenta los endpoints protegidos con JWT en las dos versiones de Spring Boot?	2 h	Swagger UI con autorización Bearer funcionando	Cierra D-12

7. Backlog arquitectónico priorizado

Usamos la misma escala del product backlog: prioridad de 1 a 10 y dimensión de 10 a 100. El orden es por prioridad y, a igual prioridad, primero la menor dimensión. Cada ítem indica la deuda o el spike del que sale y cuándo se considera terminado.

ID	Ítem	Origen	Prior.	Dim.	Terminado cuando…
A-01	Spike SP-1 y wrapper de Gradle en AuthCore	D-03	10	20	`gradlew build` funciona en una máquina limpia
A-02	GitHub Actions: compilar y probar los dos servicios en cada push y pull request	D-02	10	30	Workflow en verde en los dos repositorios
A-03	Ejecutar todas las pruebas, corregir fallos y agregar pruebas unitarias de AuthService	D-01, D-03	10	40	`mvn verify` y `gradlew build` pasan sin fallos
A-04	Credenciales del administrador inicial por variables de entorno, sin escribirlas en el log; rotar la contraseña de MySQL	D-04	9	20	Ninguna credencial en código, configuración versionada ni logs
A-05	Spike SP-2 y migraciones versionadas en los dos servicios	D-05	9	60	El esquema se crea solo con migraciones; `ddl-auto: validate`
A-06	Política de contraseñas y registro con correo según HU-1 de AuthCore	D-06	8	20	Pasan las pruebas de cada criterio de HU-1
A-07	Llevar la regla «turno vigente» al dominio (EstadoTurno) y usarla en el adaptador y en el doble	D-07	8	30	El contrato del repositorio prueba el filtro con todos los estados

ID	Ítem	Origen	Prior.	Dim.	Terminado cuando…
A-08	Unificar la numeración de HU en README, requests.http y los dos backlogs	D-11	7	20	Una sola tabla de HU citada por todos los documentos
A-09	Spike SP-5 y medición de tiempos; cierra el criterio 5 de HU-7	D-10	7	30	Informe con las cuatro operaciones por debajo de 1 s
A-10	Spike SP-6 y documentación OpenAPI	D-12	6	30	Swagger UI visible y revisado por el otro integrante
A-11	Spike SP-3 y ADR sobre la firma RS256	D-08	6	50	ADR con decisión, alcance y archivos de los dos servicios
A-12	Reintento o mensaje claro ante conflictos de concurrencia	D-13	5	30	Prueba con dos hilos que agendan el mismo horario
A-13	Spike SP-4 y ADR de revocación de tokens	D-09	5	50	ADR con decisión y alcance
A-14	Paginación en los listados	D-14	4	40	Parámetros page y size con prueba
A-15	Alinear versiones de Java y Spring Boot según SP-1	D-15	3	60	Misma plataforma, o un ADR que justifique la diferencia

8. Plan de dos sprints

Proponemos sprints de una semana, porque el calendario del semestre no da para sprints de dos. El rol de Product Owner y el de Scrum Master rotan entre los dos integrantes en cada sprint.

Sprint	Meta del sprint	Contenido	Dimensión	Roles
Sprint 1 10 – 16 nov	Base técnica confiable y segura: los dos servicios compilan y se prueban solos en cada cambio, y no queda ninguna credencial expuesta.	A-01, A-02, A-03, A-04, A-05	170	PO: Andrés SM: Jhorman
Sprint 2 17 – 23 nov	Requisitos cumplidos y demostrables: HU-1 de AuthCore y HU-7 se cumplen, la regla de estados vuelve al dominio y la API queda documentada.	A-06, A-07, A-08, A-09 (cierra HU-7), A-10	130	PO: Jhorman SM: Andrés
Después	Seguridad avanzada y escalabilidad	A-11 a A-15	230	Se replanifica en la revisión del sprint 2

- **Por qué el sprint 1 es más grande.** Sus ítems bloquean al resto: sin build e integración continua ningún otro ítem se puede dar por terminado. Si su velocidad queda por debajo de 170, A-05 pasa al sprint 2 antes que A-06.
- **Eventos.** Planificación con póker de planificación sobre la escala de dimensión; Scrum diario breve por el canal del grupo; revisión con demostración en clase; retrospectiva con una lista de mejoras para el siguiente sprint.

9. Definición de terminado

Adoptamos la propuesta común de la guía y le agregamos lo que nos faltó según la parte I: integración continua, ningún secreto y revisión en pull request.

- El código compila y todas las pruebas pasan, incluidas las de arquitectura, y el workflow de GitHub Actions está en verde (desde A-02).
- Ninguna credencial ni secreto queda en el código, la configuración versionada o los logs.
- La decisión técnica relevante quedó registrada como ADR en `docs/adr/`.
- El README refleja el estado real: qué está verificado y qué queda pendiente.
- El otro integrante revisó el cambio en un pull request antes de integrarlo a main.
- El ítem se puede demostrar en la revisión del sprint.

10. Tablero del sprint

El tablero está en Trello: trello.com/b/JqZOzIz4 («Chronos · Sistema de Turnos (Grupo 5)»; público, de solo lectura para quien no es miembro). Tiene 37 tarjetas: una guía fija con las metas de los sprints y la definición de terminado, los 15 ítems A-01 a A-15 y las 21 historias de usuario (14 del Domain Service y 7 de AuthCore). Cada tarjeta lleva en el título el ID y la dimensión, y en la descripción el origen, la prioridad y la deuda o el spike del que sale. Las historias tienen sus criterios de aceptación como lista de verificación, y los ítems A, su criterio de terminado.

Etiquetas: Deuda técnica, Spike, Historia (Domain Service o AuthCore), Sprint 1, Sprint 2 e «Implementada, falta verificar», para las historias que ya están en el código pero aún no cumplen la definición de terminado.

Product Backlog	Sprint Backlog	En curso	En revisión (PR)	Terminado
A-11 a A-15 y las HU sin planificar	Ítems del sprint vigente, en orden de prioridad	Máximo un ítem por integrante	Esperando la revisión del otro integrante	Cumple todos los puntos de la definición de terminado

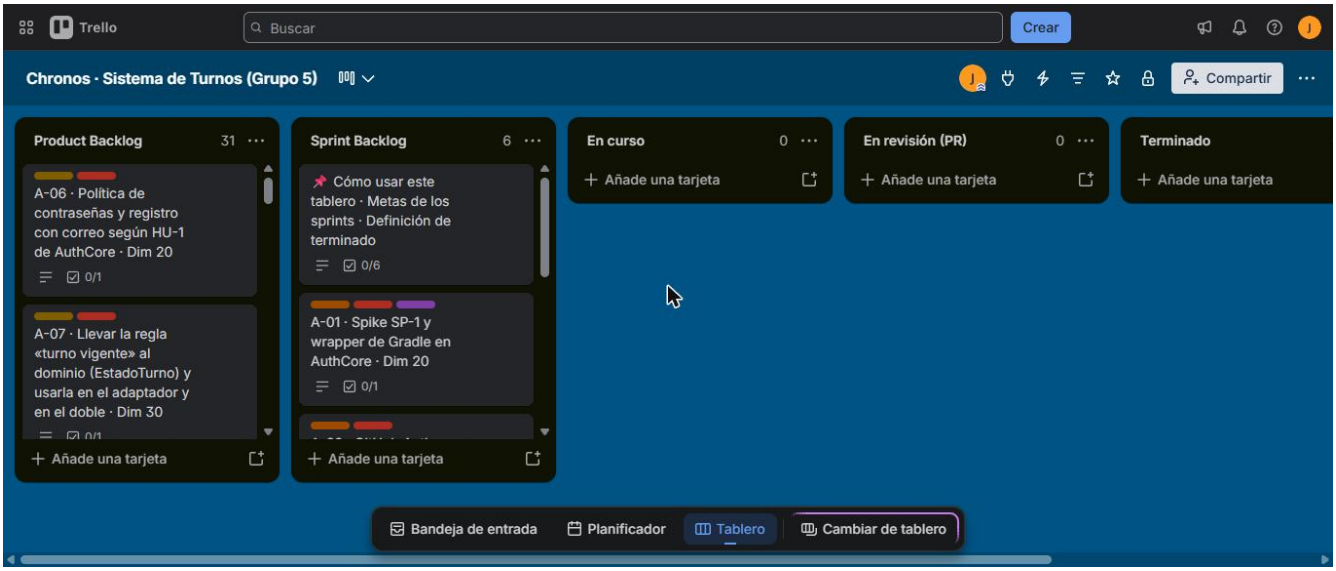


Figura 3. Tablero de Trello del proyecto al iniciar el sprint 1 (9 de noviembre de 2026).

Anexo A. Decisiones ya tomadas que se registrarán como ADR

Estas decisiones existen en el código o en el backlog, pero aún no tienen un registro propio. Se escribirán en `docs/adr/` con el formato de Nygard (2011): contexto, decisión y consecuencias.

ADR	Decisión	Contexto	Consecuencias
001	El Domain Service valida el JWT localmente con el secreto compartido (HS384, mínimo 48 bytes)	No debe llamar a AuthCore en cada petición	Rápido e independiente; el Domain Service podría fabricar tokens (D-08, se revisa en SP-3)
002	Solo el rol ADMIN opera el Domain Service	Clientes y profesionales no usan el sistema	USER recibe 403; abrir la consulta a USER exige cambiar SecurityConfig
003	Sin API externa: el turno usa los clientes y profesionales registrados	Observación del docente sobre RF-02	RF-02 depende de RF-01 y RF-06; no hay integración externa
004	Cada requisito tiene una acción y un rol; los ID son estables y los nuevos van al final	Observaciones del docente sobre RNF y roles	9 RF, 5 RNF y 14 HU; la Hoja 1 del docente sigue siendo trazable
005	La transacción es un puerto (TransaccionPort), no @Transactional	La aplicación no debe importar Spring	Los casos de uso se prueban con un doble que ejecuta el código directamente

Referencias

Barrientos Suárez, C. A. (2026). *Metodologías de desarrollo y su impacto en la arquitectura: guía de estudio, semanas 9 y 10*. Universidad de Pamplona.

Barrientos Suárez, C. A. (2026). *Domain Service de Sistema de Turnos: guía de funcionamiento, paquetes, integración con AuthCore y uso*. Universidad de Pamplona.

Beck, K. et al. (2001). *Manifiesto por el Desarrollo Ágil de Software*. agilemanifesto.org

Cunningham, W. (1992). The WyCash Portfolio Management System. *OOPSLA '92 Experience Report*.

Fowler, M. (2009). *Technical Debt Quadrant*. martinfowler.com

Kruchten, P. (1995). The 4+1 View Model of Architecture. *IEEE Software*, 12(6).

Kruchten, P. *The Rational Unified Process: An Introduction*. Addison-Wesley.

Nygard, M. (2011). *Documenting Architecture Decisions*. cognitect.com/blog

Royce, W. (1970). Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON*.

Schwaber, K. y Sutherland, J. (2020). *La Guía de Scrum*. scrumguides.org